

Super Tic-Tac-Toe as a Stochastic Reinforcement Learning Environment

ZHONG Mingqian, HOU Yuhe

Abstract

This project develops a stochastic super tic-tac-toe game as a reinforcement learning environment. The game is formalized as a centered three-level pyramid board with 96 playable squares, stochastic action realization, forfeited moves, and row, column, and diagonal winning patterns. A complete environment is implemented with action masks, stochastic move resolution, winner detection, rendering, and systematic validation tests.

Baseline experiments are conducted using a random agent and a one-step heuristic agent. Under the corrected centered pyramid geometry, the heuristic agent achieves a 98.5% win rate against the random agent over 200 games, showing that the environment can clearly reflect differences in strategy quality. A DQN-style reinforcement learning pipeline is then implemented by wrapping the two-player game into a single-agent task against a fixed random opponent. The trained DQN agent reaches a best evaluation win rate of 74.0% at episode 1200, while the last evaluation at episode 2000 reaches 60.0%. Although the learned policy remains weaker than the heuristic baseline and the training curve is non-monotonic, the results demonstrate that the corrected stochastic board-game environment is runnable, testable, extensible, and suitable for reinforcement learning experiments.

Contents

1	Phase A: Problem Formulation and Rule Interpretation	4
1.1	Problem Interpretation	4
1.2	Symbol Conventions	5
1.3	Coordinate and Indexing Scheme	5
1.4	Cross-Level Geometry and Adjacency	6
1.5	Winning Pattern Definitions	7
1.5.1	Row Patterns	7
1.5.2	Column Patterns	7
1.5.3	Diagonal Patterns	7
1.5.4	Implementation Convention for Winner Checking	7
2	Phase B: Environment Design and Implementation	8
2.1	Overall Objective of Phase B	8
2.2	Implementation Convention for Geometry	8
2.3	Geometry Construction and Static Structures	9
2.4	Neighborhood Structure and Stochastic Move Realization	9
2.5	Winning Pattern Enumeration	9
2.5.1	Row Lines	10
2.5.2	Diagonal Lines	10
2.5.3	Column Lines	10
2.5.4	Unified Winning Line Set	10
2.6	Environment Class Design	10
2.6.1	Reset	11
2.6.2	Action Mask	11
2.6.3	Step Function	11
2.7	Testing and Validation	11
2.8	Outcome of Phase B	11
3	Phase C: Baseline Agents and Matchup Evaluation	12
3.1	Objective of Phase C	12
3.2	Baseline Agent Design	12
3.2.1	Random Agent	12
3.2.2	Heuristic Agent	12
3.3	Game Execution and Evaluation Pipeline	13

3.4	Matchup Configurations	13
3.5	Experimental Results	13
3.5.1	Random vs Random	15
3.5.2	Heuristic vs Random	15
3.5.3	Heuristic vs Heuristic	15
3.6	Discussion	15
3.7	Outcome of Phase C	15
4	Phase D: TorchRL-Based DQN Training and Learning Results	16
4.1	Objective of Phase D	16
4.2	Single-Agent Training Formulation	16
4.3	Training Environment Wrapper	17
4.4	State, Action, and Reward Design	17
4.4.1	Observation	17
4.4.2	Action Space	17
4.4.3	Reward Function	17
4.5	DQN Training Pipeline	18
4.6	Training Configuration	19
4.7	Main Training Results	19
4.8	Comparison with Baselines	20
4.9	Discussion	21
4.10	Outcome of Phase D	22
5	Phase E: Interactive Game Interface	22
6	Conclusion	24

1 Phase A: Problem Formulation and Rule Interpretation

1.1 Problem Interpretation

This project considers a two-player stochastic super tic-tac-toe game and formulates it as a reinforcement learning environment. Compared with standard tic-tac-toe, this game is substantially more complex in terms of board topology, stochastic action execution, and winning conditions. Therefore, it should not be treated as a simple deterministic board game. Instead, it is more appropriately viewed as a two-player sequential decision-making problem with a discrete state space, a discrete action space, stochastic state transitions, and adversarial interaction.

The game is played by two players in alternating turns. According to the project specification, Player 1 places noughts, while Player 2 places crosses. Player 1 moves first by default. At each discrete time step, the current player selects an empty square as the intended target position. Importantly, the selected target square is not necessarily the final realized placement. In other words, the player controls an intended action, while the environment determines the realized placement according to a fixed stochastic rule.

The stochastic action mechanism is one of the defining features of this problem. Let a denote the square selected by the current player. The environment places the mark at a with probability $1/2$, and places it at each of the eight adjacent same-level squares with probability $1/16$. Therefore, an action should not be interpreted as a deterministic placement, but rather as a noisy intended control. This requires a successful strategy to consider not only the target square itself, but also the surrounding neighborhood, boundary risks, and possible conflicts with occupied squares.

If the realized position sampled by the environment is outside the board, or if the position is already occupied, the move is forfeited. A forfeited move means that the current player loses the turn, while the board state remains unchanged. As a result, the environment contains spatially nonuniform risk: boundary and corner positions are more likely to lead to out-of-board realization, while crowded regions in the middle and late game are more likely to produce occupied-square conflicts.

The terminal conditions are also more complex than those of standard tic-tac-toe. A player wins by forming four consecutive marks in a row, four consecutive marks in a column, or five consecutive marks across a diagonal. In addition, the project statement specifies that a column win must involve at least one move in a different level. This implies that a column cannot simply be interpreted as an ordinary vertical four-in-a-column pattern within a single board plane; instead, it must reflect the multi-level structure of the board. If all playable squares are occupied and no winning condition is satisfied, the game ends in a draw.

Overall, this project formalizes the game as a two-player stochastic game. An agent must select intended target squares under adversarial interaction and stochastic action realization, and must learn robust strategies despite noisy transitions. The problem combines board-geometry modeling, stochastic action resolution, and adversarial decision-making, making it a meaningful environment-design and reinforcement-learning task.

1.2 Symbol Conventions

The following symbols are used throughout the report:

- $\mathcal{P} = \{1, -1\}$ denotes the set of players, where 1 represents Player 1 and -1 represents Player 2.
- $\mathcal{L} = \{1, 2, 3\}$ denotes the set of board levels.
- $\mathcal{B}_\ell$ denotes the set of blocks in level ℓ , with

$$|\mathcal{B}_1| = 1, \quad |\mathcal{B}_2| = 2, \quad |\mathcal{B}_3| = 3.$$

- $(level, block, row, col)$ denotes the logical coordinate of a playable square.
- $(level, gr, gc)$ denotes the level-wise geometric coordinate of a playable square.
- $index \in \{0, 1, \dots, 95\}$ denotes the one-dimensional action index associated with a playable square.
- The board state is denoted by a vector or array s , where each playable square takes values in $\{-1, 0, 1\}$, representing Player 2, empty, and Player 1, respectively.
- The intended action selected by a player is denoted by a , while the realized placement is denoted by z .

1.3 Coordinate and Indexing Scheme

To represent the board state unambiguously and to connect it naturally with a discrete reinforcement learning action space, each playable square is assigned both a logical coordinate and a one-dimensional action index. This dual representation preserves the hierarchical board structure while supporting state encoding, action masking, and winning-line enumeration.

The board consists of six 4×4 blocks arranged into three levels. Level 1 contains one 4×4 block, Level 2 contains two horizontally adjacent 4×4 blocks, and Level 3 contains three horizontally adjacent 4×4 blocks. Therefore, the total number of playable squares is

$$6 \times 4 \times 4 = 96.$$

Thus, the action space can be naturally represented as a discrete set of size 96, where each action corresponds to selecting a target playable square.

A square is logically represented by

$$(level, block, row, col),$$

where $level \in \{1, 2, 3\}$, $block$ denotes the block index within a level, and $row, col \in \{0, 1, 2, 3\}$ denote the local row and column coordinates inside a 4×4 block. The admissible block indices are

$$block \in \begin{cases} \{0\}, & level = 1, \\ \{0, 1\}, & level = 2, \\ \{0, 1, 2\}, & level = 3. \end{cases}$$

Each playable square is further mapped to a unique one-dimensional index

$$index \in \{0, 1, \dots, 95\}.$$

The index is assigned in lexicographic order over $(level, block, row, col)$. With level offsets

$$offset(1) = 0, \quad offset(2) = 16, \quad offset(3) = 48,$$

the index of a square is defined as

$$index = offset(level) + 16 \cdot block + 4 \cdot row + col.$$

This indexing scheme is reversible. Given an index, one can identify the level, subtract the corresponding offset, and recover the block, row, and column. Therefore, the implementation maintains mappings between indices and logical coordinates.

1.4 Cross-Level Geometry and Adjacency

The logical coordinate is sufficient for structural identification, but it is not enough to determine geometric continuity. Winning patterns depend on spatial continuity across blocks and potentially across levels. Therefore, each playable square is also embedded into a centered global geometric coordinate. To match the pyramid-shaped board in the assignment figure, all three levels are placed in a common width-12 coordinate system. Level 1 is centered above Level 2, and Level 2 is centered above Level 3.

Let $h(level)$ denote the horizontal offset of each level in this centered layout. The offsets are defined as

$$h(1) = 4, \quad h(2) = 2, \quad h(3) = 0.$$

Each playable square is then represented by the level-wise geometric coordinate

$$(level, gr, gc),$$

where

$$gr = row, \quad gc = h(level) + 4 \cdot block + col.$$

Thus, the global column ranges are

$$\text{Level 1 : } gc \in \{4, 5, 6, 7\},$$

$$\text{Level 2 : } gc \in \{2, 3, \dots, 9\},$$

$$\text{Level 3 : } gc \in \{0, 1, \dots, 11\}.$$

This creates a centered pyramid embedding of the board that matches the assignment figure.

The same-level adjacent squares are defined using the Moore neighborhood. For a selected square $(level, gr, gc)$, its eight adjacent candidate positions are

$$\{(level, gr + \Delta r, gc + \Delta c) : \Delta r, \Delta c \in \{-1, 0, 1\}, (\Delta r, \Delta c) \neq (0, 0)\}.$$

Adjacency is defined only within the same level. If a candidate position is outside the playable region of the current level, it is treated as outside the board. If the stochastic realization samples such a position, the move is forfeited rather than being redistributed to valid positions.

1.5 Winning Pattern Definitions

The winning patterns are defined using geometric continuity rather than index adjacency. A sequence of squares forms a valid winning pattern only if it is geometrically continuous under the coordinate system described above.

1.5.1 Row Patterns

A row win is defined as four horizontally consecutive playable squares within the same level. Formally, if a player occupies

$$(level, gr, gc), \quad (level, gr, gc + 1), \quad (level, gr, gc + 2), \quad (level, gr, gc + 3),$$

and all four squares are valid, then the player forms a row win. This definition allows row patterns to cross block boundaries as long as the squares remain horizontally continuous in the level-wise geometric coordinate.

1.5.2 Column Patterns

A column win requires four geometrically continuous vertical squares and must involve at least two different levels. This condition reflects the project statement that a column win must include a move in a different level. Therefore, ordinary vertical four-in-a-column patterns within a single level are not treated as column wins. In the implementation, candidate column patterns are enumerated in a stacked display coordinate system and are retained only if they contain four valid squares and involve at least two distinct levels.

1.5.3 Diagonal Patterns

A diagonal win requires five geometrically consecutive playable squares along a diagonal direction. Since a single 4×4 block contains diagonals of length at most four, a valid length-five diagonal must extend across a larger geometric region, potentially crossing block boundaries. The two basic diagonal directions are

$$(\Delta r, \Delta c) = (1, 1) \quad \text{and} \quad (\Delta r, \Delta c) = (1, -1).$$

A player wins diagonally if the player occupies five valid squares that form a continuous sequence in one of these directions.

1.5.4 Implementation Convention for Winner Checking

In the implementation, all legal winning lines are pre-enumerated during initialization. The set of winning lines is denoted by

$$\mathcal{W} = \{W_1, W_2, \dots, W_M\},$$

where each W_i is a sequence of indices corresponding to a valid row, column, or diagonal pattern. During gameplay, after each effective placement, the environment checks whether

there exists a line $W_i \in \mathcal{W}$ such that all positions in W_i are occupied by the same player. If so, that player is declared the winner.

This pre-enumeration strategy separates rule specification from runtime checking, simplifies debugging, and allows the correctness of winning patterns to be verified through tests.

2 Phase B: Environment Design and Implementation

The goal of Phase B is to transform the rule interpretation and geometric formalization into a runnable, testable reinforcement learning environment. This phase implements the static board geometry, action and state spaces, stochastic move realization, winner detection, draw detection, and systematic validation tests. The resulting `SuperTicTacToeEnv` serves as the foundation for baseline agents and DQN training.

2.1 Overall Objective of Phase B

Phase A focuses on rule clarification and formal specification, while Phase B turns the specification into executable code. The main objectives are:

1. to construct static geometric structures and index mappings for all playable squares;
2. to implement core environment interfaces, including `reset()`, `step(action)`, legality checking, observation construction, and action masking;
3. to implement stochastic move realization according to the required probabilities;
4. to implement winner and draw detection and validate them through targeted tests.

2.2 Implementation Convention for Geometry

The game rules are implemented in `geometry.py`. The module does not redefine the rules; rather, it converts the formal specification into static data structures that can be queried efficiently during environment execution.

Each square maintains three coordinate representations: the logical coordinate $(level, block, row, col)$, the level-wise geometric coordinate $(level, gr, gc)$, and the stacked display coordinate (br, bc) . Logical coordinates recover the hierarchical block structure; level-wise geometric coordinates support same-level neighborhoods and horizontal continuity; stacked display coordinates support length-five diagonals and cross-level column patterns. In the corrected implementation, both the geometric coordinate and the stacked display coordinate use the centered pyramid offsets $h(1) = 4$, $h(2) = 2$, and $h(3) = 0$.

In the implementation, the following static structures are constructed:

- `index_to_lbrc` and `lbrc_to_index`;
- `index_to_geom` and `geom_to_index`;
- `index_to_display` and `display_to_index`;
- `neighbors`;
- `row_lines`, `diag_lines`, and `column_lines`;
- `winning_lines`.

The final implementation contains 96 playable squares, 60 row lines, 60 diagonal lines, and 36 column lines, giving a total of 156 winning lines. The environment’s `check_winner()` function is entirely based on this pre-enumerated set.

2.3 Geometry Construction and Static Structures

The `geometry.py` module is responsible for constructing the static geometric information required by the environment. It provides index mappings, centered coordinate conversions, neighborhood tables, and winning-line collections. These structures allow the environment to switch between different coordinate systems during action execution, neighbor lookup, winning-line enumeration, and rendering.

2.4 Neighborhood Structure and Stochastic Move Realization

The stochastic move mechanism is one of the most important features of the environment. A player first selects an intended action, and the environment then samples the realized position. Specifically,

$$P(\text{realized action} = a) = \frac{1}{2},$$

where a is the intended target square. For each of the eight same-level neighbors n_i ,

$$P(\text{realized action} = n_i) = \frac{1}{16}, \quad i = 1, \dots, 8.$$

To support this mechanism, `geometry.py` precomputes a `neighbors` table. Each entry maps an index to a list of eight neighbor candidates. If a neighbor direction is outside the board, the corresponding entry is set to `None`. The function `resolve_action()` then performs the following steps:

1. construct the candidate list consisting of the original target and its eight same-level neighbors;
2. sample one candidate using the fixed probability distribution;
3. if the sampled candidate is `None`, forfeit the move;
4. if the sampled candidate is occupied, forfeit the move;
5. otherwise, place the current player’s mark at the realized position.

This implementation preserves the original probability distribution and does not redistribute probability mass from invalid outcomes. For example, a corner square has five out-of-board adjacent directions, so the out-of-board forfeiture probability is 5/16.

2.5 Winning Pattern Enumeration

After constructing the static coordinate mappings, all legal winning patterns are pre-enumerated. The environment separately generates `row_lines`, `diag_lines`, and `column_lines`, and then merges them into `winning_lines`. During gameplay, `check_winner()` only needs to iterate over this set and check whether any winning line is fully occupied by one player.

2.5.1 Row Lines

`row_lines` are generated using the level-wise geometric coordinate. For each row in each level, a sliding window of length four is used to enumerate horizontally continuous playable squares. The final implementation generates 60 row lines:

$$4 \times (1 + 5 + 9) = 60.$$

2.5.2 Diagonal Lines

`diag_lines` are generated using the stacked display coordinate. The algorithm enumerates length-five candidates in both diagonal directions and retains only those whose positions all correspond to valid playable squares. The final implementation generates 60 diagonal lines under the centered pyramid geometry.

2.5.3 Column Lines

`column_lines` are also generated in the stacked display coordinate. The algorithm first enumerates vertically consecutive length-four candidates and then keeps only those involving at least two distinct levels. This excludes ordinary within-level vertical lines. The final implementation generates 36 column lines.

2.5.4 Unified Winning Line Set

The final number of winning lines is

$$|\text{winning_lines}| = |\text{row_lines}| + |\text{diag_lines}| + |\text{column_lines}| = 60 + 60 + 36 = 156.$$

This design keeps the runtime environment logic simple: winner checking becomes a fixed traversal over a static list of valid patterns.

2.6 Environment Class Design

The main environment class is `SuperTicTacToeEnv`. It uses a flat board array of length 96:

$$\text{board}[i] \in \{-1, 0, 1\},$$

where -1 represents Player 2, 0 represents an empty square, and 1 represents Player 1. The environment maintains the following variables:

- `current_player`;
- `done`;
- `winner`;
- `move_count`;
- `last_action`;
- `last_realized_action`;
- `last_move_forfeited`.

2.6.1 Reset

The `reset()` function clears the board, sets the current player to Player 1, resets all terminal and bookkeeping variables, and returns the initial observation. The observation consists of the 96-dimensional board state and one additional dimension for the current player, giving a total dimension of 97.

2.6.2 Action Mask

The environment implements `get_action_mask()`, which returns a Boolean vector of length 96. Empty squares are marked as `True`, while occupied squares are marked as `False`. This mask is used later by the DQN agent to avoid illegal actions.

2.6.3 Step Function

The `step(action)` function is the core environment interface. It checks action legality, resolves the stochastic realization, handles forfeited moves, updates the board if a valid placement occurs, checks for a winner or draw, switches the current player if necessary, and returns the new observation and auxiliary information. This design explicitly separates the intended action from the realized action, accurately reflecting the stochastic execution rule.

2.7 Testing and Validation

A manual test script, `run_tests.py`, was developed to validate the environment. The tests cover:

- reset behavior;
- action masks;
- legal and illegal actions;
- successful and forfeited steps;
- stochastic realization probabilities;
- boundary forfeiture probability;
- row, diagonal, and column winner detection;
- false-positive cases for incomplete winning lines;
- observation and bookkeeping consistency.

All tests passed, indicating that the environment is not only executable but also consistent with the formal rule interpretation.

2.8 Outcome of Phase B

Phase B successfully transforms the formal rule specification into a verified environment. The main outputs are:

- a complete board geometry model with coordinate mappings;
- the stochastic move and forfeiture mechanism;

- row, diagonal, and column winning-line enumeration;
- the `SuperTicTacToeEnv` environment class;
- systematic tests covering the key environment logic.

At the end of Phase B, the project has a validated stochastic two-player board-game environment ready for baseline evaluation and reinforcement learning training.

3 Phase C: Baseline Agents and Matchup Evaluation

After completing the environment implementation, the project proceeds to baseline agent construction and matchup evaluation. The purpose of this phase is not to train an RL agent immediately, but to first establish simple and interpretable baselines. These baselines help verify whether the environment behaves reasonably in complete games and provide comparison points for later DQN training.

3.1 Objective of Phase C

The objectives of Phase C are:

1. to verify that complete games terminate properly and exhibit reasonable dynamics;
2. to construct interpretable baseline agents;
3. to establish a standardized evaluation pipeline for later RL experiments.

3.2 Baseline Agent Design

3.2.1 Random Agent

The `RandomAgent` uniformly samples from all legal actions. If the legal action set in state s is denoted by $\mathcal{A}_{\text{legal}}(s)$, then

$$\pi_{\text{rand}}(a \mid s) = \begin{cases} \frac{1}{|\mathcal{A}_{\text{legal}}(s)|}, & a \in \mathcal{A}_{\text{legal}}(s), \\ 0, & \text{otherwise.} \end{cases}$$

This agent serves as the minimal performance baseline.

3.2.2 Heuristic Agent

The `HeuristicAgent` uses a simple one-step deterministic approximation. Its policy is:

1. if the current player has an immediate winning move, take it;
2. otherwise, if the opponent has an immediate winning threat, block it;
3. otherwise, choose a random legal action.

The heuristic agent has been corrected to reason from the perspective of the current player, rather than assuming a fixed player identity. This ensures that it behaves consistently whether it plays as Player 1 or Player 2. Since the actual environment is stochastic, this heuristic is not optimal; however, it is a strong and interpretable baseline.

3.3 Game Execution and Evaluation Pipeline

The function `play_one_game()` runs a complete game between two agents. It resets the environment, selects the appropriate agent according to `current_player`, executes actions until termination, and returns summary statistics such as winner, draw status, move count, total turns, and optional game history.

The function `evaluate_matchup()` repeats this process for N games and reports:

- Player 1 win count and win rate;
- Player 2 win count and win rate;
- draw count and draw rate;
- average move count;
- average number of turns.

Both `move_count` and total turns are recorded because forfeited moves may increase the number of turns without increasing the number of successful placements.

3.4 Matchup Configurations

Three baseline matchups are evaluated:

1. **Random vs Random;**
2. **Heuristic vs Random;**
3. **Heuristic vs Heuristic.**

Each matchup is evaluated over 200 games with fixed seed settings for reproducibility.

3.5 Experimental Results

The baseline results are shown in Table 1.

Table 1: Baseline matchup results under the centered pyramid geometry					
Matchup	P1 Win	P2 Win	Draw	Avg Move Count	Avg Turns
Random vs Random	0.545	0.455	0.000	48.85	64.80
Heuristic vs Random	0.985	0.015	0.000	32.20	40.80
Heuristic vs Heuristic	0.515	0.485	0.000	33.67	43.28

To visualize these results, Figure 1 compares win rates across the three matchups, while Figure 2 compares average game lengths.

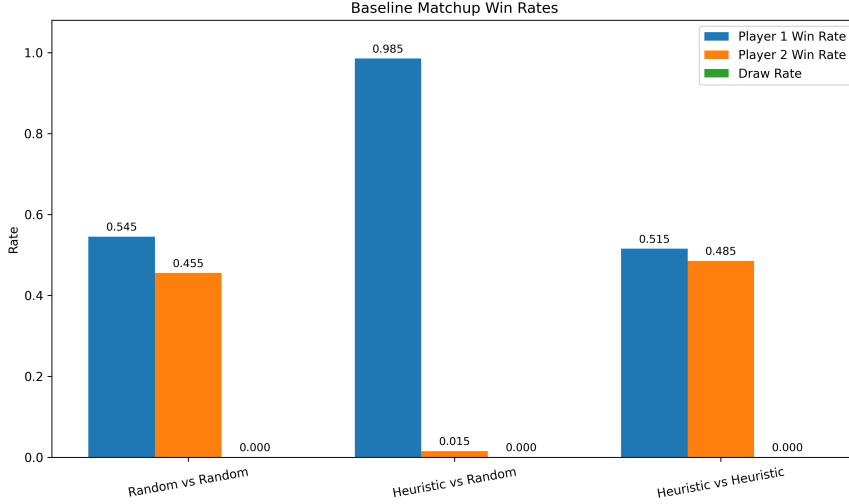


Figure 1: Win rate comparison for different baseline matchups.

Figure 1 shows that Random vs Random is relatively balanced, with a mild first-player advantage. In Heuristic vs Random, the heuristic agent achieves a win rate of 98.5%, showing that even a simple one-step heuristic can strongly outperform random play under the corrected centered pyramid geometry. In Heuristic vs Heuristic, Player 1 wins 51.5% of the games, indicating that the corrected geometry leads to a nearly symmetric matchup when both players use the same heuristic policy.

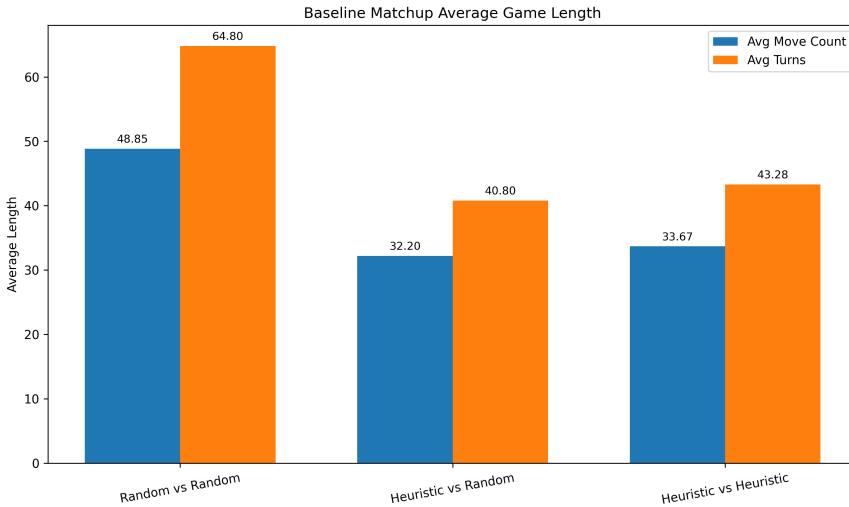


Figure 2: Average game length comparison for different baseline matchups.

Figure 2 shows that Random vs Random produces longer games, with an average move count of 48.85 and an average turn count of 64.80. In contrast, Heuristic vs Random and Heuristic vs Heuristic are much shorter, with average turn counts of 40.80 and 43.28, respectively. This indicates that heuristic play accelerates the formation or prevention of decisive patterns.

3.5.1 Random vs Random

In Random vs Random, Player 1 wins 54.5% of games and Player 2 wins 45.5%. No draws occur. This suggests a mild first-player advantage, which is reasonable for a turn-based board game. The average move count is 48.85 and the average turn count is 64.80. The average turn count is larger than the average move count because some moves are forfeited.

3.5.2 Heuristic vs Random

In Heuristic vs Random, the heuristic player wins 98.5% of the games. This demonstrates that the environment can clearly distinguish between strategies of different quality, and that stochastic action realization does not completely obscure strategic advantage. The average move count is 32.20 and the average turn count is 40.80, which are much smaller than those in Random vs Random.

3.5.3 Heuristic vs Heuristic

When both players use the heuristic policy, Player 1 wins 51.5% of the games and Player 2 wins 48.5%. This result is close to symmetric, indicating that the heuristic policy does not introduce a strong imbalance under the corrected centered pyramid layout. The average move count is 33.67 and the average turn count is 43.28, which are slightly larger than in Heuristic vs Random but still much shorter than in Random vs Random.

3.6 Discussion

The baseline experiments serve two purposes. First, they provide direct comparison points for the later DQN agent. Second, they dynamically validate the environment beyond single-step unit tests. The results show that games terminate reliably, strategy quality is reflected in outcome statistics, and the stochastic mechanism does not prevent meaningful strategic differentiation.

The heuristic policy is still shallow: it only performs one-step deterministic evaluation and does not reason over long-term opportunities or the full stochastic realization distribution. Therefore, although it is much stronger than random play, it should not be treated as an optimal policy. This leaves room for learning-based agents to develop more robust strategies.

3.7 Outcome of Phase C

Phase C completes:

- the implementation of `RandomAgent` and `HeuristicAgent`;
- the correction of the heuristic player’s perspective logic;
- the implementation of one-game and multi-game evaluation pipelines;
- three 200-game baseline matchups;
- a set of clear performance baselines for reinforcement learning.

4 Phase D: TorchRL-Based DQN Training and Learning Results

After correcting the board geometry to the centered pyramid layout, the project further evaluates whether a value-based reinforcement learning agent can learn useful strategies in the stochastic super tic-tac-toe environment. Unlike the baseline agents in Phase C, which are manually designed, Phase D trains a DQN-style agent through repeated interaction with the environment. The original two-player game is converted into a single-agent learning problem by fixing the opponent as a random agent.

The goal of this phase is not to claim that the DQN agent already solves the game, but to verify that the corrected environment can support a complete reinforcement learning pipeline. The implemented training pipeline includes a Q-network, a target network, epsilon-greedy exploration with legal-action masking, replay-buffer sampling, Double DQN-style target computation, periodic evaluation, and best-checkpoint saving.

4.1 Objective of Phase D

The objectives of Phase D are threefold:

1. to wrap the corrected two-player stochastic game into a single-agent training environment;
2. to implement a DQN/Double DQN-style training pipeline with action masking and replay-buffer sampling;
3. to compare the learned policy with the baseline results obtained under the centered pyramid geometry.

This phase therefore serves as a bridge between environment construction and learning-based decision making. It validates whether the environment provides meaningful learning signals and whether a neural value function can exploit the game structure better than random action selection.

4.2 Single-Agent Training Formulation

The original environment is a two-player alternating stochastic game. For DQN training, it is transformed into a single-agent task by fixing the learning agent as Player 1 and using a `RandomAgent` as Player 2. Each call to the training wrapper first executes the learning agent’s intended action. If the game does not terminate after this action, the random opponent automatically executes one move. The resulting state is then returned to the learning agent.

Under this formulation, the learning agent always acts from Player 1’s perspective. This simplification allows the two-player game to be treated as a standard single-agent reinforcement learning problem against a fixed opponent. Although this is not yet self-play, it is sufficient for validating the DQN pipeline and for evaluating whether the agent can learn a policy that performs better than random play.

4.3 Training Environment Wrapper

The wrapper environment `SingleAgentVsRandomEnv` is built on top of `SuperTicTacToeEnv`. Its interaction process is summarized as follows:

1. reset the base environment and return the initial observation for Player 1;
2. receive an intended action from the learning agent;
3. execute the learning agent’s stochastic move in the base environment;
4. if the game terminates, return the terminal transition immediately;
5. otherwise, let the random opponent execute one stochastic move;
6. return the updated observation, reward, termination flag, and action mask.

This wrapper preserves the stochastic action realization rule, the forfeited-move mechanism, and the centered pyramid winning-line structure. Therefore, the DQN agent is trained on the same corrected environment used in the baseline experiments.

4.4 State, Action, and Reward Design

4.4.1 Observation

The observation is a 97-dimensional vector. The first 96 entries represent the flattened board state, and the final entry stores the current player indicator. Each board entry takes values in

$$\{-1, 0, 1\},$$

where 1 represents Player 1, -1 represents Player 2, and 0 represents an empty square. Although the single-agent wrapper usually returns states only when it is Player 1’s turn, the current-player feature is retained for consistency with the base environment.

4.4.2 Action Space

The action space contains 96 discrete intended target squares:

$$a \in \{0, 1, \dots, 95\}.$$

Each action represents the square selected by the agent before stochastic realization. Because some squares may already be occupied, the environment also returns an action mask indicating which actions are currently legal. This mask is used during both exploration and greedy action selection.

4.4.3 Reward Function

The reward is defined from Player 1’s perspective:

$$r = \begin{cases} +1, & \text{if Player 1 wins,} \\ -1, & \text{if Player 1 loses,} \\ 0, & \text{if the game draws or the transition is non-terminal.} \end{cases}$$

This terminal-only reward preserves the zero-sum nature of the game. However, it also makes the learning problem sparse, since most intermediate transitions receive zero reward. This sparsity is one important reason why the DQN performance remains non-monotonic and unstable.

4.5 DQN Training Pipeline

The DQN training pipeline consists of the following components:

- a feedforward Q-network `QNetwork`;
- a target network `target_net`;
- a replay buffer;
- epsilon-greedy exploration;
- legal-action masking;
- Double DQN-style target computation;
- periodic evaluation and checkpoint saving.

The Q-network maps the 97-dimensional observation to 96 Q-values, one for each possible intended action. The network architecture contains two hidden layers with 256 units each:

$$97 \rightarrow 256 \rightarrow 256 \rightarrow 96.$$

During action selection, illegal actions are masked by assigning them a very small Q-value. Therefore, both random exploration and greedy exploitation are restricted to legal empty squares.

Each transition stored in the replay buffer has the form

$$(s, a, r, s', done, mask, mask'),$$

where *mask* and *mask'* denote the legal-action masks before and after the transition. Once the replay buffer contains at least `min_replay_size` transitions, mini-batches are sampled for optimization.

The target value follows a Double DQN-style update. The online network first selects the best legal next action under the next-state mask:

$$a^* = \arg \max_{a' \in \mathcal{A}_{legal}(s')} Q_{\theta}(s', a').$$

Then the target network estimates the value of this selected action:

$$y = r + \gamma(1 - done)Q_{\theta^-}(s', a^*).$$

The online network is optimized by minimizing the mean squared Bellman error:

$$\mathcal{L}(\theta) = (Q_{\theta}(s, a) - y)^2.$$

The implementation evaluates the policy every 100 episodes. At each evaluation checkpoint, the policy is tested against a random opponent without exploration. The training script keeps track of the best evaluation win rate. Whenever a new best evaluation result is obtained, the corresponding model parameters are stored as the current best checkpoint. At the end of training, the best checkpoint is saved to `checkpoints/dqn_final.pt`. Therefore, `dqn_final.pt` corresponds to the best evaluated model rather than simply the model at the last training episode.

4.6 Training Configuration

The main training configuration is:

- total training episodes: 2000;
- maximum steps per episode: 300;
- discount factor: $\gamma = 0.99$;
- learning rate: 10^{-3} ;
- replay buffer size: 50,000;
- minimum replay size before optimization: 1,000;
- batch size: 64;
- target network update frequency: 500 environment steps;
- epsilon starts from 1.0 and decays to 0.05;
- evaluation is performed every 100 episodes over 50 evaluation games.

This configuration is intended as a preliminary but complete training setup. It is sufficient to validate the end-to-end reinforcement learning pipeline, although it is not yet an extensively tuned configuration for this stochastic board game.

4.7 Main Training Results

The DQN agent was evaluated periodically against the random opponent. The evaluation results are summarized in Table 2.

Table 2: Evaluation results during DQN training under the corrected centered geometry

Episode	Win Rate	Loss Rate	Avg Return
100	0.720	0.280	0.440
200	0.700	0.300	0.400
300	0.580	0.420	0.160
400	0.520	0.480	0.040
500	0.720	0.280	0.440
600	0.600	0.400	0.200
700	0.620	0.380	0.240
800	0.500	0.500	0.000
900	0.460	0.540	-0.080
1000	0.500	0.500	0.000
1100	0.600	0.400	0.200
1200	0.740	0.260	0.480
1300	0.440	0.560	-0.120
1400	0.500	0.500	0.000
1500	0.640	0.360	0.280
1600	0.580	0.420	0.160
1700	0.520	0.480	0.040
1800	0.480	0.520	-0.040
1900	0.460	0.540	-0.080
2000	0.600	0.400	0.200

The best observed evaluation occurs at episode 1200, where the agent reaches a win rate of 0.740, a loss rate of 0.260, and an average return of 0.480. The final evaluation at episode 2000 gives a win rate of 0.600. Since the training script saves the best evaluated model, the saved checkpoint `checkpoints/dqn.final.pt` corresponds to the episode-1200 model rather than the episode-2000 model.



Figure 3: DQN training performance against the random opponent.

Figure 3 shows that the DQN agent learns useful behavior at several stages of training. The win rate is above the Random vs Random first-player baseline of 0.545 at many evaluation checkpoints, especially at episodes 100, 200, 500, 700, 1200, 1500, 1600, and 2000. However, the curve is not monotonic. The win rate drops below or near the random baseline at several checkpoints, such as episodes 800, 900, 1300, 1400, 1800, and 1900. This indicates that the current DQN setup can learn useful policies but does not yet provide stable monotonic improvement.

4.8 Comparison with Baselines

Figure 4 compares the DQN results with the baseline agents. Under the corrected centered pyramid geometry, Random vs Random gives a Player 1 win rate of 0.545, while Heuristic vs Random gives a Player 1 win rate of 0.985. The best DQN evaluation reaches 0.740 at episode 1200, and the last evaluation at episode 2000 reaches 0.600.

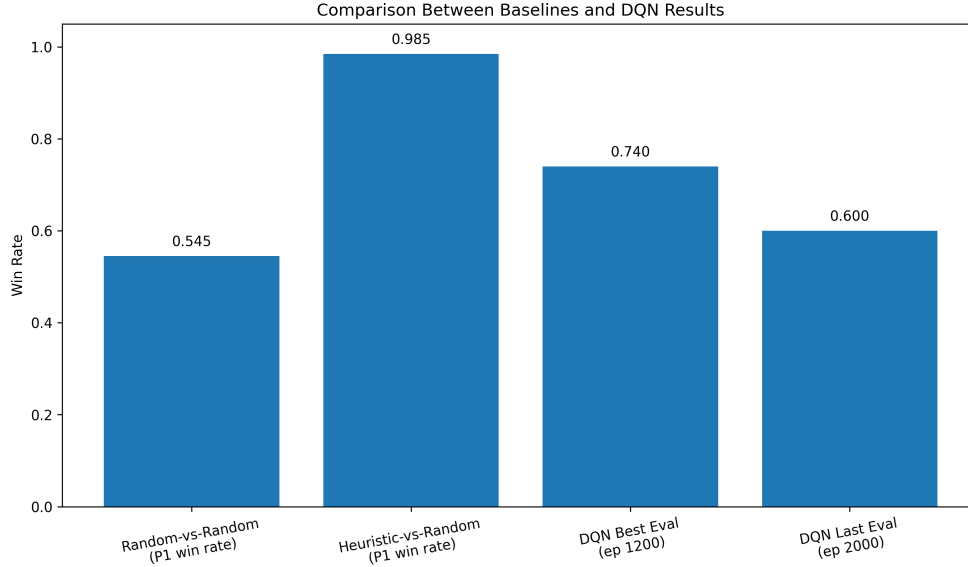


Figure 4: Comparison between baseline strategies and DQN results.

The comparison shows that the best DQN policy performs clearly better than random play. In particular, the best DQN win rate of 0.740 is substantially higher than the Random vs Random first-player baseline of 0.545. However, the DQN agent remains weaker than the one-step heuristic agent, which achieves a win rate of 0.985 against the random opponent. This result is reasonable because the heuristic agent directly uses the winner-detection logic to identify immediate winning moves and blocking moves, whereas the DQN agent must infer value structure from sparse rewards and stochastic transitions.

4.9 Discussion

The DQN experiment provides several observations. First, the corrected environment can be used as a valid reinforcement learning environment: the agent can interact with it, collect transitions, store replay data, apply legal-action masking, and update the Q-network through Bellman backups. Second, the model can learn nontrivial behavior, as shown by the best evaluation win rate of 0.740. Third, the learning process remains unstable, since strong intermediate checkpoints are followed by weaker evaluations.

There are several possible reasons for this instability:

1. **Sparse reward.** Most non-terminal transitions receive zero reward, so the agent receives limited feedback before the end of the game.
2. **Stochastic action realization.** The selected intended action may not be the final realized move, which makes credit assignment more difficult.
3. **Large action space.** The agent must choose among 96 possible intended actions.
4. **Complex geometry.** The centered pyramid board contains row, column, and diagonal winning patterns across a non-rectangular board structure.
5. **Fixed random opponent.** Training only against a random opponent may not provide sufficiently rich strategic pressure.

Therefore, the current DQN result should be interpreted as a successful preliminary learning result rather than a final strong game-playing agent. Future improvements may

include reward shaping, slower epsilon decay, learning-rate tuning, prioritized replay, stronger opponents, or self-play.

4.10 Outcome of Phase D

Phase D completes the following tasks:

- the corrected environment is wrapped into a single-agent learning task against a random opponent;
- a DQN/Double DQN-style training pipeline is implemented;
- action masking is used to restrict both exploration and exploitation to legal moves;
- replay-buffer sampling and target-network updates are incorporated;
- periodic evaluation is performed against a fixed random opponent;
- the best evaluated checkpoint is saved as `checkpoints/dqn_final.pt`;
- the trained agent reaches a best observed evaluation win rate of 0.740.

Overall, Phase D demonstrates that the corrected stochastic super tic-tac-toe environment can support reinforcement learning experiments. Although the current DQN agent does not approach the heuristic baseline, it performs meaningfully better than random play at its best checkpoint and provides a clear foundation for further algorithmic improvement.

5 Phase E: Interactive Game Interface

In addition to the terminal-based game watching script, this project also provides a lightweight interactive interface implemented with `Streamlit`. The purpose of this interface is to make the stochastic environment directly observable and playable. Although a graphical interface is not required by the original assignment, it serves as a useful demonstration tool for inspecting the stochastic move mechanism, the agent interaction process, and the final game outcome.

The updated interface is consistent with the corrected centered pyramid geometry. In particular, each level is rendered inside a global width-12 layout: Level 1 is centered with width 4, Level 2 is centered with width 8, and Level 3 occupies the full width 12. This makes the visual board layout consistent with the formal geometry used in `geometry.py` and the environment.

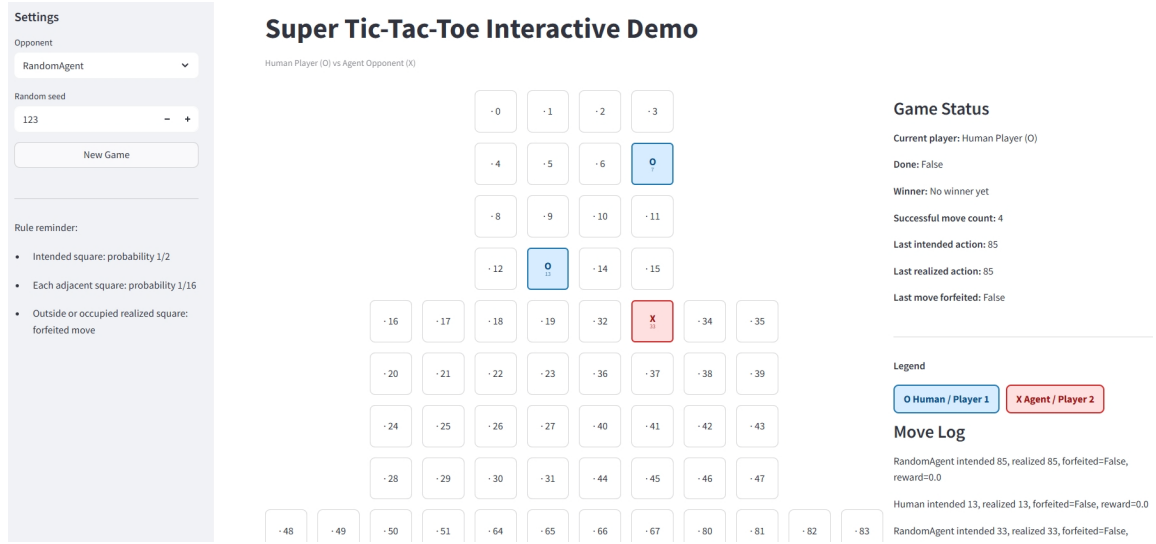


Figure 5: Lightweight interactive interface for playing the stochastic super tic-tac-toe game under the centered pyramid layout.

As shown in Figure 5, the interface is divided into three main regions. The left sidebar contains the game settings, including the opponent type and the random seed. In the updated version, the opponent can be selected from **RandomAgent**, **HeuristicAgent**, and **DQNAgent**. This allows the same interface to be used not only for baseline demonstrations, but also for testing a trained DQN policy. When **DQNAgent** is selected, an additional text input is displayed for specifying the model checkpoint path, with the default value set to `checkpoints/dqn_final.pt`. The random seed controls the stochastic action realization process and makes the game behavior reproducible. A **New Game** button is provided to reset the environment and start a new game.

The sidebar also displays a short rule reminder. The intended square is realized with probability $1/2$, each adjacent same-level square is realized with probability $1/16$, and a move is forfeited if the realized square is outside the board or already occupied. This reminder is important because the user does not directly control the final placement; the clicked square is only the intended action, while the realized action is sampled by the environment.

The center panel displays the board level by level using the centered pyramid layout. Each playable square is represented as a clickable cell with its corresponding one-dimensional action index. Empty cells are shown as clickable buttons marked with a dot, while occupied cells are rendered as colored cards instead of disabled buttons. In the current visualization, O denotes the human player or Player 1, and X denotes the agent opponent or Player 2. The colored-card design makes occupied squares more visible than ordinary disabled buttons and improves readability during gameplay.

The board rendering uses a fixed maximum width of 12 columns. For each level, the interface computes the horizontal padding

$$\text{left_pad} = \frac{12 - \text{width}(\text{level})}{2},$$

so that Level 1, Level 2, and Level 3 are displayed with widths 4, 8, and 12 respectively. This implementation reproduces the centered pyramid shape in the interactive page and

avoids the earlier left-aligned visualization.

The right panel shows the current game status. It includes the current player, whether the game has terminated, the winner, the number of successful placements, the last intended action, the last realized action, and whether the last move was forfeited. These quantities are essential for this environment because of the stochastic transition rule. By displaying both the intended action and the realized action, the interface makes it easy to verify whether stochastic move resolution is being applied correctly.

The right panel also includes a legend and a move log. The legend explains the color encoding for **O** and **X**. The move log records recent actions, including the intended action, realized action, forfeiture status, and reward information. This allows the user to inspect a game trajectory step by step rather than only observing the final board state.

To play the game, the user first selects an opponent and a random seed from the sidebar. If the selected opponent is **DQNAgent**, the user may also specify the checkpoint file to load. After clicking **New Game**, the user plays as **O** by selecting an empty cell on the board. The environment then resolves the stochastic move. If the game has not terminated, the selected opponent automatically plays as **X**. The board, game status, and move log are then updated. This process repeats until a player wins or the game ends in a draw.

Overall, the interactive interface provides an intuitive way to demonstrate that the implemented environment is not only usable for batch evaluation and DQN training, but also playable and inspectable at the level of individual decisions. Compared with the terminal-only visualization, this page more clearly illustrates the centered pyramid board geometry, the distinction between intended and realized actions, the possibility of forfeited moves, and the behavior of different opponent policies, including the trained DQN agent.

6 Conclusion

This project completes a full reinforcement learning workflow for a stochastic super tic-tac-toe game, including rule interpretation, environment implementation, baseline evaluation, and preliminary DQN training. Compared with standard tic-tac-toe, this task has a more complex board structure, a larger action space, stochastic move realization, and cross-level winning patterns. Therefore, the central challenge is not only to train an agent, but also to build a clear, stable, testable, and reproducible environment.

First, the game rules are formalized systematically. The board is modeled as a three-level centered pyramid structure with six 4×4 blocks and 96 playable squares. Logical coordinates, level-wise geometric coordinates, and stacked display coordinates are introduced to represent the board and enumerate winning patterns. Row, diagonal, and column wins are defined consistently and implemented through pre-enumerated winning lines.

Second, the **SuperTicTacToeEnv** environment is implemented. It supports reset, step execution, action legality checking, action masks, stochastic move realization, forfeited moves, winner detection, draw detection, and rendering. The stochastic action mechanism follows the project specification exactly: the intended target is realized with probability $1/2$, and each of the eight same-level adjacent candidates is realized with probability $1/16$. Invalid or occupied realized positions lead to forfeited moves. Systematic tests

confirm the correctness of initialization, action masks, stochastic realization, boundary behavior, and winner detection.

Third, two baseline agents are implemented and evaluated. The **RandomAgent** serves as a minimal baseline, while the **HeuristicAgent** uses a one-step deterministic approximation to choose winning moves, block immediate threats, or otherwise play randomly. After correcting the player-perspective logic, the heuristic agent achieves a 98.5% win rate against the random agent over 200 games under the corrected centered pyramid geometry. This demonstrates that the environment can clearly reflect strategy quality.

Finally, a DQN-style reinforcement learning pipeline is built. The two-player game is wrapped into a single-agent task against a fixed random opponent. The training pipeline uses a Q-network, target network, epsilon-greedy exploration with action masking, replay-buffer sampling, Double DQN-style target computation, periodic evaluation, and best-checkpoint saving. The best evaluated DQN checkpoint reaches a win rate of 74.0% at episode 1200 against the random opponent, while the last evaluation at episode 2000 reaches 60.0%. These results show that the agent can learn meaningful behavior from the stochastic environment, although the learning curve remains non-monotonic and the policy is still weaker than the one-step heuristic baseline.

Overall, the project establishes a complete and extensible experimental framework for reinforcement learning in a stochastic board-game environment. Although the current DQN policy is not yet a strong strategy, the corrected environment, baseline evaluations, interactive demonstration, and training pipeline provide a solid foundation for future work. Possible extensions include improved observation encoding, reward shaping, lower learning rates, slower exploration decay, longer training, prioritized replay, stronger opponents, and self-play.